Sr. no	Name of Practical's
1	Write program to demonstrate the following aspects of signal processing on suitable data 1. Upsampling and downsampling on Image/speech signal 2. Fast Fourier Transform to compute DFT
2	Write program to perform the following on signal 1. Create a triangle signal and plot a 3-period segment. 2. For a given signal, plot the segment and compute the correlation between them.
3	Write program to demonstrate the following aspects of signal on sound/image data 1. Convolution operation 2. Template Matching
4	Write program to implement point/pixel intensity transformations such as 1. Log and Power-law transformations 2. Contrast adjustments 3. Histogram equalization 4. Thresholding, and halftoning operations
5	Write a program to apply various enhancements on images using image derivatives by implementing Gradient and Laplacian operations.
6	Write a program to implement linear and nonlinear noise smoothing on suitable image or sound signal.
7	Write a program to apply various image enhancement using image derivatives by implementing smoothing, sharpening, and unsharp masking filters for generating suitable images for specific application requirements
8	Write a program to Apply edge detection techniques such as Sobel and Canny to extract meaningful information from the given image samples
9	Write the program to implement various morphological image processing techniques.
10	Write the program to extract image features by implementing methods like corner and blob detectors, HoG and Haar features
11	Write the program to apply segmentation for detecting lines, circles, and other shapes/ objects. Also, implement edge-based and region-based segmentation.

Practical 1

Aim:-Write program to demonstrate the following aspects of signal processing on suitable data

1. Upsampling and downsampling on Image/speech signal

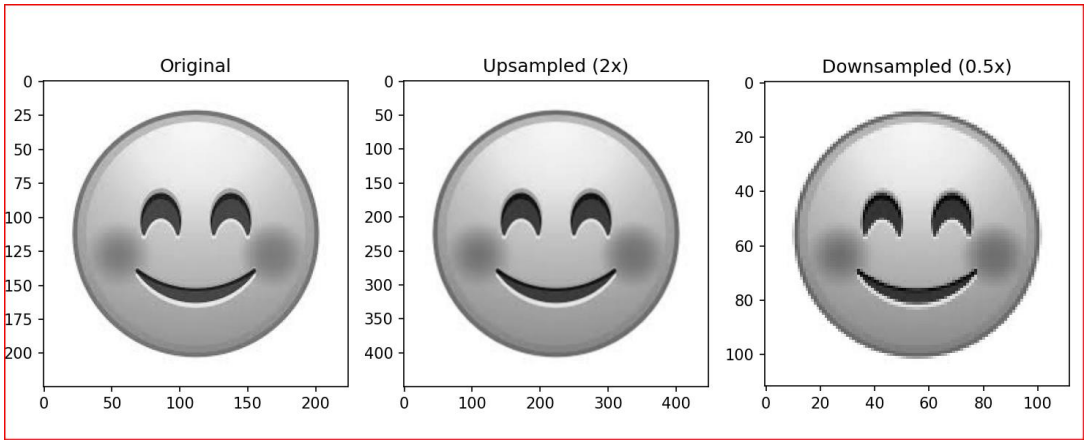
2. Fast Fourier Transform to compute DFT

Image_Signal.py

```
import cv2

import numpy as np

import matplotlib.pyplot as plt


# Read grayscale image

img = cv2.imread("C:\\Users\\Shreyas S Sutar\\Desktop\MSC CS SEM-I\\download.jpg",
cv2.IMREAD_GRAYSCALE)
```

```
# Upsampling (2x)
upsampled = cv2.resize(img, None, fx=2, fy=2, interpolation=cv2.INTER_LINEAR)

# Downsampling (0.5x)
downsampled = cv2.resize(img, None, fx=0.5, fy=0.5, interpolation=cv2.INTER_AREA)

# Display images
plt.figure(figsize=(10,6))

plt.subplot(1,3,1)
plt.title("Original")
plt.imshow(img, cmap='gray')

plt.subplot(1,3,2)
plt.title("Upsampled (2x)")
plt.imshow(upsampled, cmap='gray')

plt.subplot(1,3,3)
plt.title("Downsampled (0.5x)")
plt.imshow(downsampled, cmap='gray')

plt.tight_layout()
plt.show()
```

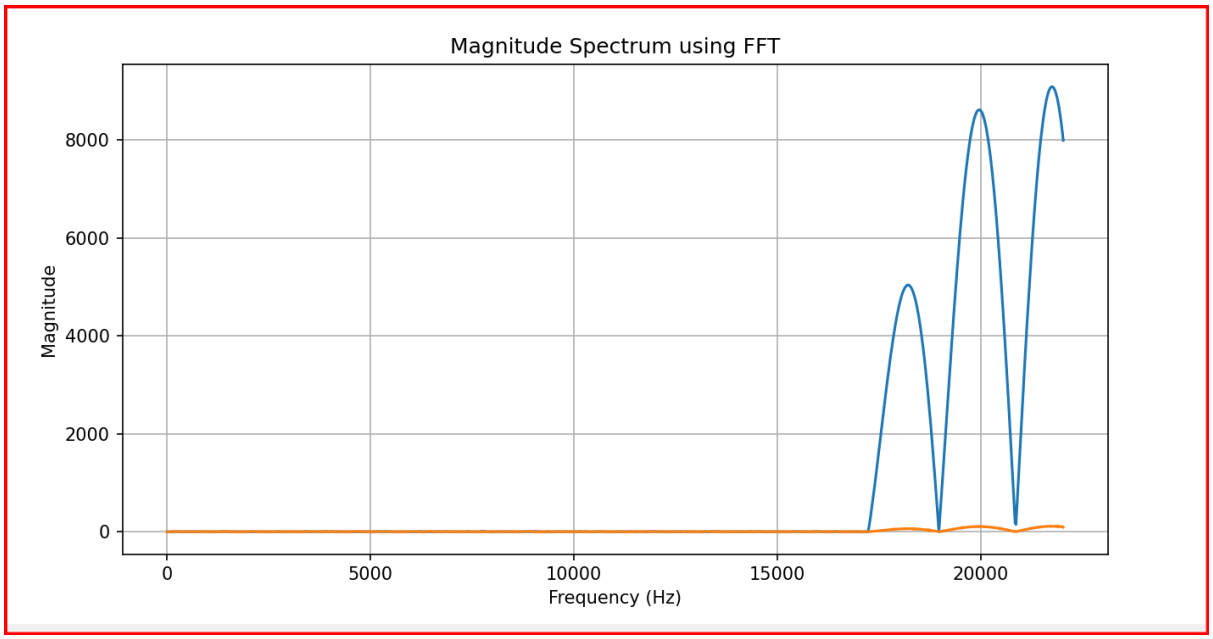
OUTPUT:-



Fast_Fourier_Transform.py

```
import numpy as np

import matplotlib.pyplot as plt

from scipy.io import wavfile


# Load speech signal

fs, speech = wavfile.read("C:\\Users\\Shreyas S Sutar\\Desktop\\MSC CS SEM-I\\file_example_WAV_1MG.wav")


# Take first 2048 samples for analysis

signal_segment = speech[:2048]


# Compute FFT

fft_values = np.fft.fft(signal_segment)


# Frequency axis

freq = np.fft.fftfreq(len(signal_segment), 1/fs)


# Magnitude Spectrum

magnitude = np.abs(fft_values)


# Plot
```

```
plt.figure(figsize=(10,5))
plt.plot(freq[:1024], magnitude[:1024])
plt.title("Magnitude Spectrum using FFT")
plt.xlabel("Frequency (Hz)")
plt.ylabel("Magnitude")
plt.grid()
plt.show()
```



Practical 2

Aim:-Write program to perform the following on signal

1. Create a triangle signal and plot a 3-period segment.

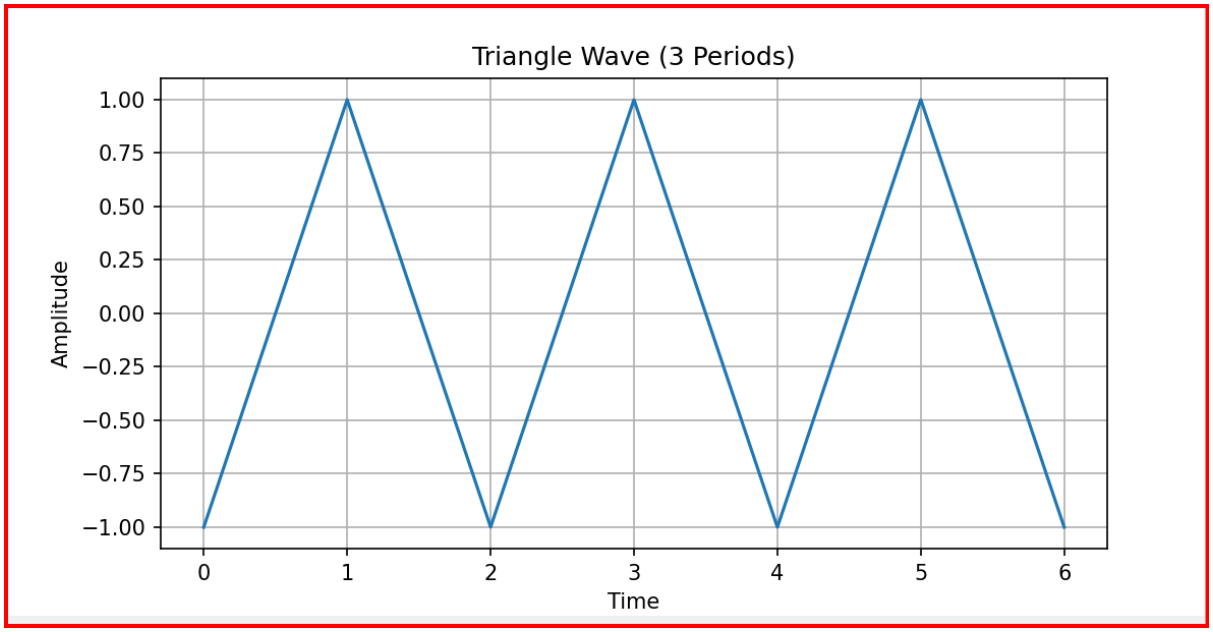
2. For a given signal, plot the segment and compute the correlation between them.

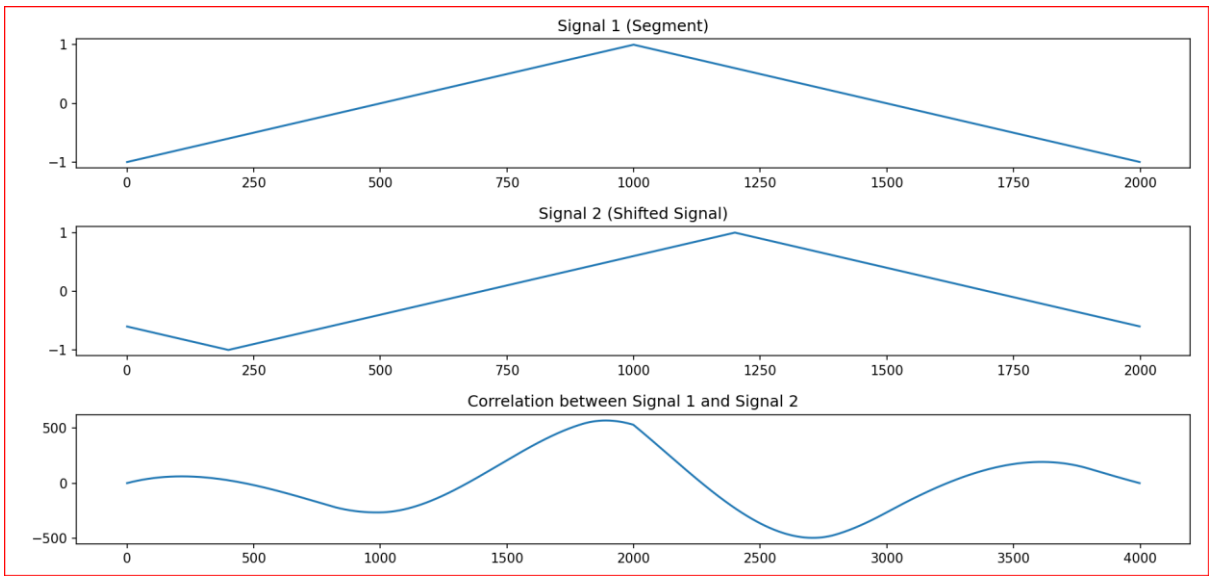
Triangle_Signal.py

```
import numpy as np
import matplotlib.pyplot as plt

# --------- PART 1: Triangle Signal ---------

# Parameters
A = 1         # Amplitude
T = 2         # Period
fs = 1000      # Sampling frequency
t = np.linspace(0, 3*T, 3*T*fs)   # 3-period time axis

# Triangle wave formula
triangle = A * (2*np.abs(2*(t/T - np.floor(t/T + 0.5))) - 1)

# Plot 3 periods
plt.figure(figsize=(8,4))
plt.plot(t, triangle)
plt.title("Triangle Wave (3 Periods)")
plt.xlabel("Time")
plt.ylabel("Amplitude")
plt.grid()
plt.show()

# --------- PART 2: Correlation ---------

# Create a second signal (shifted version)
shift = 200
signal1 = triangle[:2000]            # Original segment
signal2 = np.roll(signal1, shift)      # Shifted version

# Compute correlation
```

```
correlation = np.correlate(signal1, signal2, mode='full')

# Plot signals
plt.figure(figsize=(10,6))

plt.subplot(3,1,1)
plt.plot(signal1)
plt.title("Signal 1 (Segment)")

plt.subplot(3,1,2)
plt.plot(signal2)
plt.title("Signal 2 (Shifted Signal)")

plt.subplot(3,1,3)
plt.plot(correlation)
plt.title("Correlation between Signal 1 and Signal 2")

plt.tight_layout()
plt.show()

print("Maximum Correlation Value:", np.max(correlation))
```

Output:





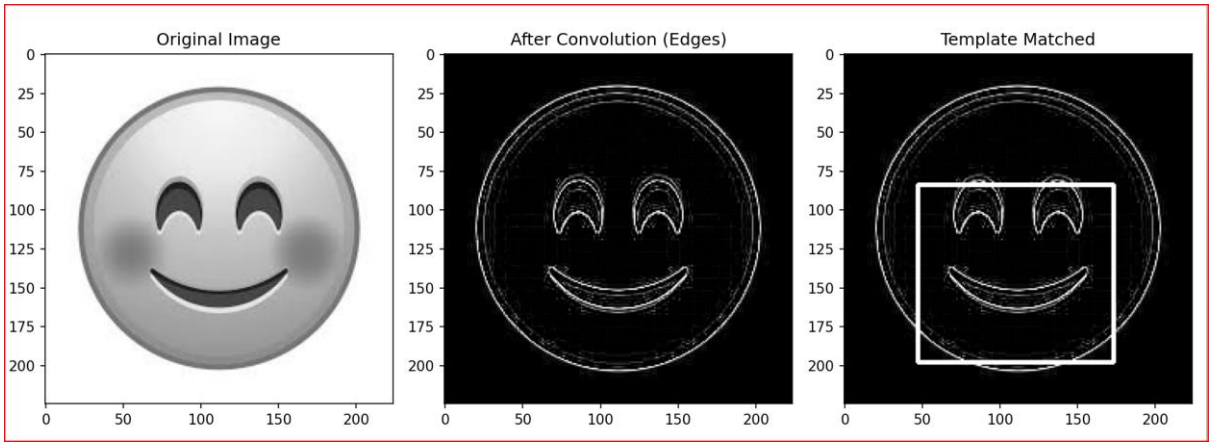
Practical 3

Aim:-Write program to demonstrate the following aspects of signal on sound/image data

1. Convolution operation 2. Template Matching

```
import cv2

import numpy as np

import matplotlib.pyplot as plt


# -------------------------

# Step 1: Load Image

# -------------------------

img = cv2.imread("C:\\Users\\Shreyas S Sutar\\Desktop\MSC CS SEM-I\\download.jpg",
cv2.IMREAD_GRAYSCALE)


# -------------------------

# Step 2: Convolution (Edge Detection)

# -------------------------

# Define an edge detection kernel

kernel = np.array([[-1,-1,-1],

                   [-1, 8,-1],

                   [-1,-1,-1]])

convolved_img = cv2.filter2D(img, -1, kernel)


# -------------------------

# Step 3: Template Matching

# -------------------------

# Load template (smaller part of the image)

template = cv2.imread("C:\\Users\\Shreyas S Sutar\\Desktop\MSC CS SEM-I\\Small_Part.jpg",
cv2.IMREAD_GRAYSCALE)


# Perform template matching

res = cv2.matchTemplate(convolved_img, template, cv2.TM_CCOEFF_NORMED)

min_val, max_val, min_loc, max_loc = cv2.minMaxLoc(res)


# Draw rectangle around matched region
```

```
top_left = max_loc
h, w = template.shape
bottom_right = (top_left[0] + w, top_left[1] + h)
matched_img = convolved_img.copy()
cv2.rectangle(matched_img, top_left, bottom_right, 255, 2)

# -------------------------
# Step 4: Display Results
# -------------------------
plt.figure(figsize=(12,5))

plt.subplot(1,3,1)
plt.title("Original Image")
plt.imshow(img, cmap='gray')

plt.subplot(1,3,2)
plt.title("After Convolution (Edges)")
plt.imshow(convolved_img, cmap='gray')

plt.subplot(1,3,3)
plt.title("Template Matched")
plt.imshow(matched_img, cmap='gray')

plt.tight_layout()
plt.show()
```



Practical 4

Write program to implement point/pixel intensity transformations such as

1. Log and Power-law transformations

2. Contrast adjustments

3. Histogram equalization

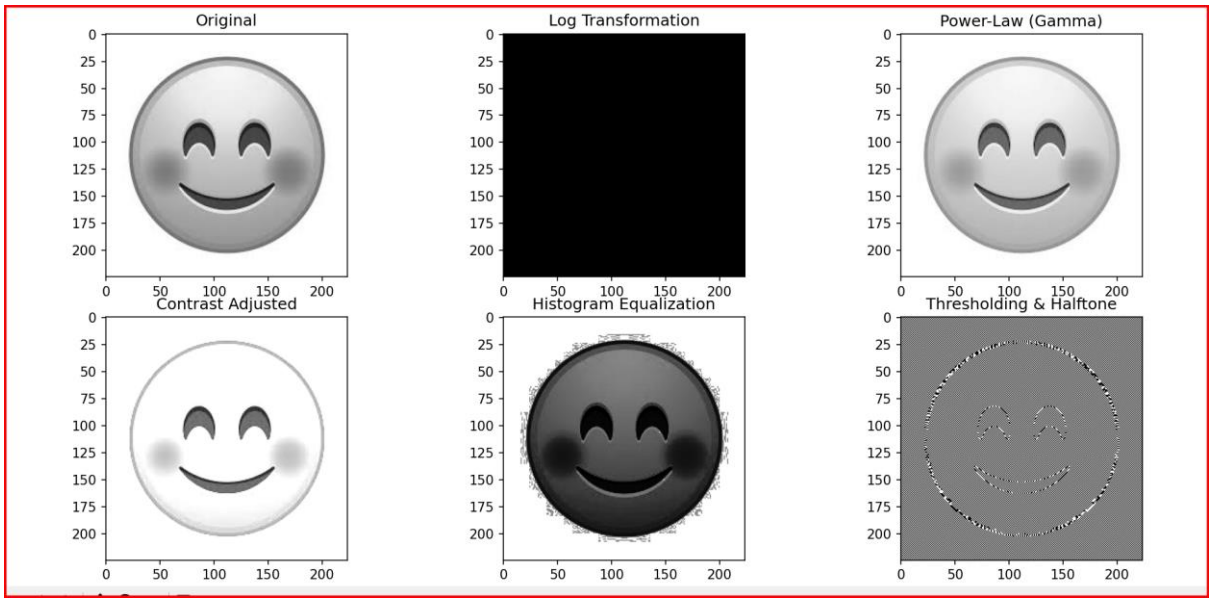
4. Thresholding, and halftoning operations

```
import cv2

import numpy as np

import matplotlib.pyplot as plt


# -------------------------

# Step 1: Load grayscale image

# -------------------------

img = cv2.imread("C:\\Users\\Shreyas S Sutar\\Desktop\MSC CS SEM-I\\download.jpg",
cv2.IMREAD_GRAYSCALE)


# -------------------------

# 1. Log Transformation

# -------------------------

c = 255 / np.log(1 + np.max(img))
```

```
log_transformed = c * np.log(1 + img.astype(np.float64))
log_transformed = np.array(log_transformed, dtype=np.uint8)

# -------------------------
# 2. Power-Law (Gamma) Transformation
# -------------------------
gamma = 0.5  # gamma < 1 brightens the image
power_law = np.array(255 * (img / 255) ** gamma, dtype=np.uint8)

# -------------------------
# 3. Contrast Adjustment (Linear Stretching)
# -------------------------
a = 1.5  # gain
b = 0    # bias
contrast_img = cv2.convertScaleAbs(img, alpha=a, beta=b)

# -------------------------
# 4. Histogram Equalization
# -------------------------
hist_eq = cv2.equalizeHist(img)

# -------------------------
# 5. Thresholding
# -------------------------
# Simple binary threshold
_, thresh_img = cv2.threshold(img, 128, 255, cv2.THRESH_BINARY)

# -------------------------
# 6. Halftoning (Simple Dithering)
# -------------------------
# Create a checkerboard halftone by thresholding with a pattern
```

```
rows, cols = img.shape
halftone = np.zeros_like(img)
for i in range(rows):
    for j in range(cols):
        if (i+j) % 2 == 0:
            halftone[i,j] = 255 if img[i,j] > 127 else 0
        else:
            halftone[i,j] = 0 if img[i,j] > 127 else 255

# -------------------------
# Step 7: Display Results
# -------------------------
plt.figure(figsize=(12,8))

plt.subplot(2,3,1)
plt.title("Original")
plt.imshow(img, cmap='gray')

plt.subplot(2,3,2)
plt.title("Log Transformation")
plt.imshow(log_transformed, cmap='gray')

plt.subplot(2,3,3)
plt.title("Power-Law (Gamma)")
plt.imshow(power_law, cmap='gray')

plt.subplot(2,3,4)
plt.title("Contrast Adjusted")
plt.imshow(contrast_img, cmap='gray')

plt.subplot(2,3,5)
```

```
plt.title("Histogram Equalization")
plt.imshow(hist_eq, cmap='gray')

plt.subplot(2,3,6)
plt.title("Thresholding & Halftone")
plt.imshow(halftone, cmap='gray')

plt.tight_layout()
plt.show()
```

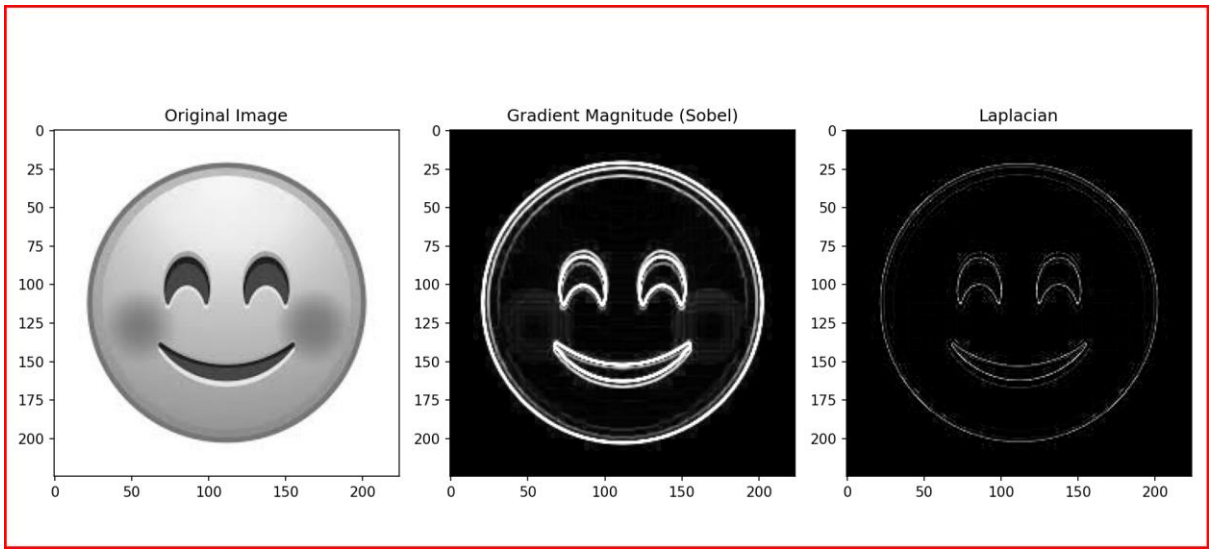


Practical 5

Write a program to apply various enhancements on images using image derivatives by implementing Gradient and Laplacian operations.

```
import cv2

import numpy as np

import matplotlib.pyplot as plt


# -------------------------

# Step 1: Load grayscale image

# -------------------------

img = cv2.imread("C:\\Users\\Shreyas S Sutar\\Desktop\MSC CS SEM-I\\download.jpg",
cv2.IMREAD_GRAYSCALE)


# -------------------------

# Step 2: Gradient Operations (Sobel)

# -------------------------

# Gradient in X direction

grad_x = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)

# Gradient in Y direction

grad_y = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)

# Combined Gradient magnitude

gradient_magnitude = cv2.magnitude(grad_x, grad_y)

gradient_magnitude = np.uint8(np.clip(gradient_magnitude, 0, 255))


# -------------------------

# Step 3: Laplacian Operation

# -------------------------

laplacian = cv2.Laplacian(img, cv2.CV_64F)

laplacian = np.uint8(np.clip(laplacian, 0, 255))


# -------------------------
```

```
# Step 4: Display Results
# -------------------------
plt.figure(figsize=(12,6))

plt.subplot(1,3,1)
plt.title("Original Image")
plt.imshow(img, cmap='gray')

plt.subplot(1,3,2)
plt.title("Gradient Magnitude (Sobel)")
plt.imshow(gradient_magnitude, cmap='gray')

plt.subplot(1,3,3)
plt.title("Laplacian")
plt.imshow(laplacian, cmap='gray')

plt.tight_layout()
plt.show()
```



Practical 6

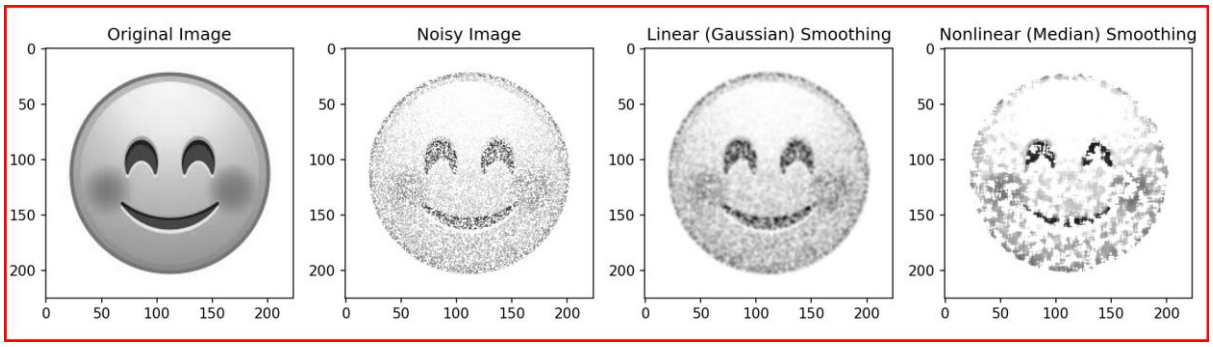
Write a program to implement linear and nonlinear noise smoothing on suitable image or sound signal.

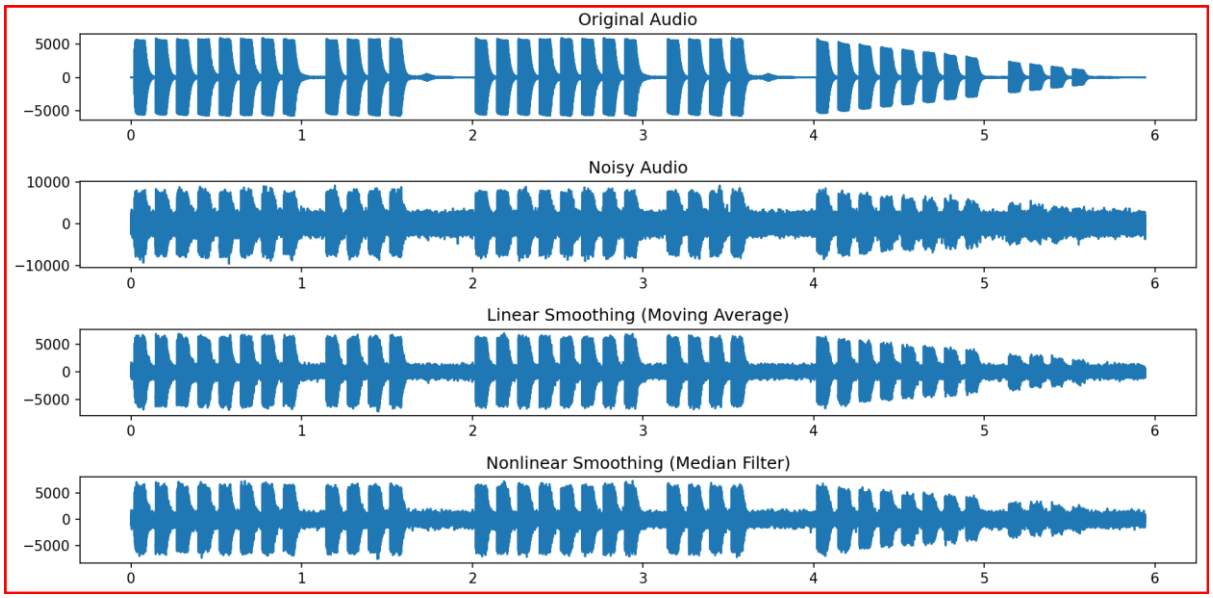
```
import cv2

import numpy as np

import matplotlib.pyplot as plt

from scipy.io import wavfile

from scipy import signal


# -------------------------

# PART 1: IMAGE NOISE SMOOTHING

# -------------------------


# Load grayscale image

img = cv2.imread("C:\\Users\\Shreyas S Sutar\\Desktop\MSC CS SEM-I\\download.jpg",
cv2.IMREAD_GRAYSCALE)


# Add Gaussian noise to the image

noise = np.random.normal(0, 25, img.shape).astype(np.uint8)

noisy_img = cv2.add(img, noise)


# --- Linear Smoothing (Gaussian Filter) ---

linear_smoothed = cv2.GaussianBlur(noisy_img, (5,5), 0)


# --- Nonlinear Smoothing (Median Filter) ---

nonlinear_smoothed = cv2.medianBlur(noisy_img, 5)


# Display image results

plt.figure(figsize=(12,6))

plt.subplot(1,4,1)

plt.title("Original Image")
```

```
plt.imshow(img, cmap='gray')

plt.subplot(1,4,2)
plt.title("Noisy Image")
plt.imshow(noisy_img, cmap='gray')

plt.subplot(1,4,3)
plt.title("Linear (Gaussian) Smoothing")
plt.imshow(linear_smoothed, cmap='gray')

plt.subplot(1,4,4)
plt.title("Nonlinear (Median) Smoothing")
plt.imshow(nonlinear_smoothed, cmap='gray')

plt.tight_layout()
plt.show()

# -------------------------
# PART 2: AUDIO NOISE SMOOTHING
# -------------------------

# Load audio file
fs, audio = wavfile.read("C:\\Users\\Shreyas S Sutar\\Desktop\\MSC CS SEM-I\\file_example_WAV_1MG.wav")

# Take mono if stereo
if len(audio.shape) > 1:
    audio = audio[:,0]

# Add Gaussian noise
noise_audio = audio + np.random.normal(0, 1000, audio.shape).astype(np.int16)
```

```
# --- Linear Smoothing (Moving Average Filter) ---
kernel_size = 5
linear_audio = np.convolve(noise_audio, np.ones(kernel_size)/kernel_size,
mode='same').astype(np.int16)

# --- Nonlinear Smoothing (Median Filter) ---
nonlinear_audio = signal.medfilt(noise_audio, kernel_size=5).astype(np.int16)

# Plot audio signals
t = np.arange(len(audio)) / fs
plt.figure(figsize=(12,6))
plt.subplot(4,1,1)
plt.plot(t, audio)
plt.title("Original Audio")

plt.subplot(4,1,2)
plt.plot(t, noise_audio)
plt.title("Noisy Audio")

plt.subplot(4,1,3)
plt.plot(t, linear_audio)
plt.title("Linear Smoothing (Moving Average)")

plt.subplot(4,1,4)
plt.plot(t, nonlinear_audio)
plt.title("Nonlinear Smoothing (Median Filter)")

plt.tight_layout()
plt.show()
```

OUTPUT:


Original Image
Noisy Image
Linear (Gaussian) Smoothing
Nonlinear (Median) Smoothing

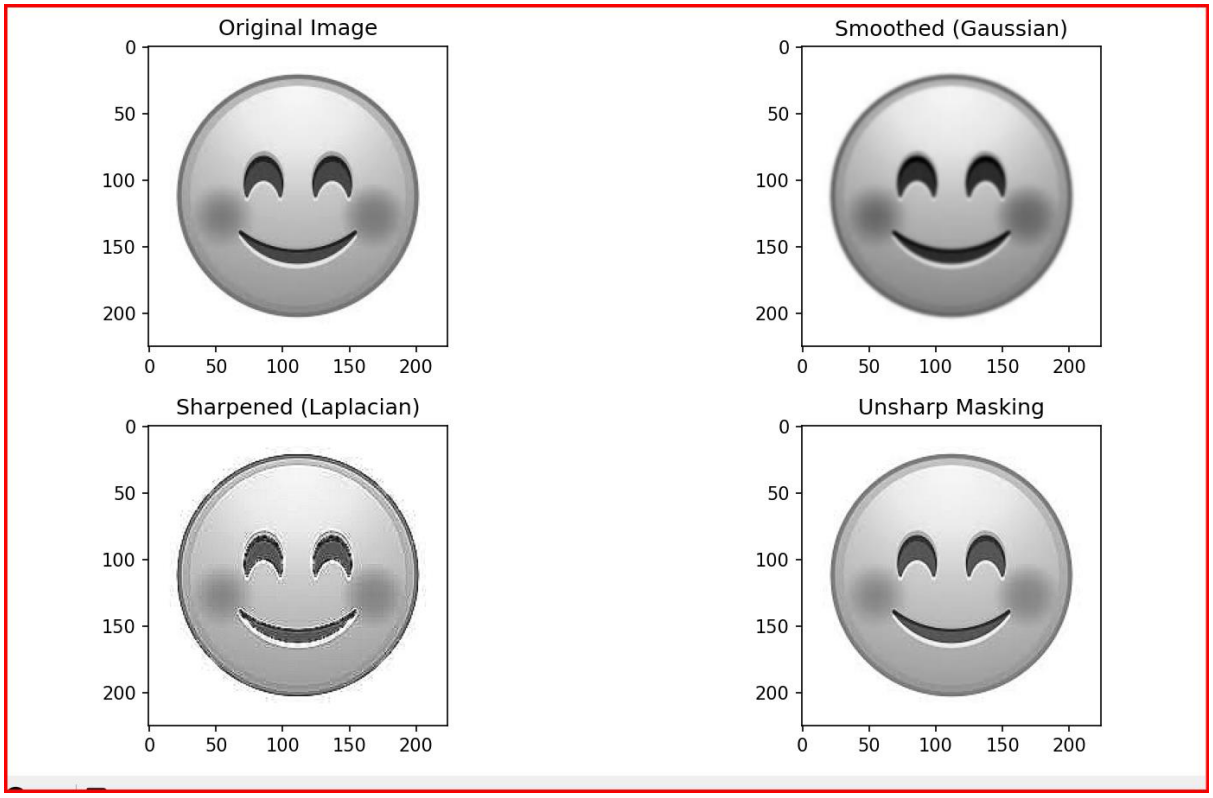

Original Audio
Noisy Audio
Linear Smoothing (Moving Average)
Nonlinear Smoothing (Median Filter)

Practical 7

Write a program to apply various image enhancement using image derivatives by implementing smoothing, sharpening, and unsharp masking filters for generating suitable images for specific application requirements

```
import cv2

import numpy as np

import matplotlib.pyplot as plt


# -------------------------

# Step 1: Load grayscale image

# -------------------------

img = cv2.imread("C:\\Users\\Shreyas S Sutar\\Desktop\MSC CS SEM-I\\download.jpg",
cv2.IMREAD_GRAYSCALE)


# -------------------------

# Step 2: Smoothing (Gaussian Blur)

# -------------------------

smoothed = cv2.GaussianBlur(img, (5,5), 0)


# -------------------------

# Step 3: Sharpening (using Laplacian)

# -------------------------

laplacian = cv2.Laplacian(img, cv2.CV_64F)

sharpened = cv2.convertScaleAbs(img - laplacian)


# -------------------------

# Step 4: Unsharp Masking

# -------------------------

# Step 4a: Blur the image

blur = cv2.GaussianBlur(img, (5,5), 0)

# Step 4b: Create mask (original - blurred)
```

```
mask = cv2.subtract(img, blur)
# Step 4c: Add mask back to original to enhance edges
unsharp = cv2.addWeighted(img, 1.5, blur, -0.5, 0)

# -------------------------
# Step 5: Display Results
# -------------------------
plt.figure(figsize=(12,6))

plt.subplot(2,2,1)
plt.title("Original Image")
plt.imshow(img, cmap='gray')

plt.subplot(2,2,2)
plt.title("Smoothed (Gaussian)")
plt.imshow(smoothed, cmap='gray')

plt.subplot(2,2,3)
plt.title("Sharpened (Laplacian)")
plt.imshow(sharpened, cmap='gray')

plt.subplot(2,2,4)
plt.title("Unsharp Masking")
plt.imshow(unsharp, cmap='gray')

plt.tight_layout()
plt.show()
```

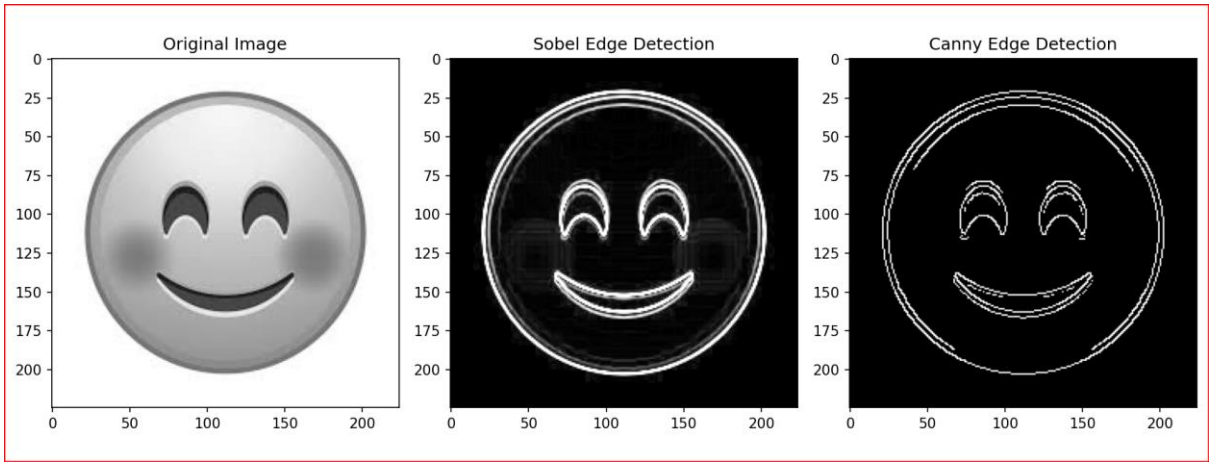

Original Image
Smoothed (Gaussian)
Sharpened (Laplacian)
Unsharp Masking
0
50
100
150
200

Practical 8

Write a program to Apply edge detection techniques such as Sobel and Canny to extract meaningful information from the given image samples

```
import cv2

import numpy as np

import matplotlib.pyplot as plt


# -------------------------

# Step 1: Load grayscale image

# -------------------------

img = cv2.imread("C:\\Users\\Shreyas S Sutar\\Desktop\MSC CS SEM-I\\download.jpg",
cv2.IMREAD_GRAYSCALE)


# -------------------------

# Step 2: Sobel Edge Detection

# -------------------------

# Gradient in X direction

sobel_x = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)

# Gradient in Y direction

sobel_y = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)

# Combined gradient magnitude

sobel_combined = cv2.magnitude(sobel_x, sobel_y)

sobel_combined = np.uint8(np.clip(sobel_combined, 0, 255))


# -------------------------

# Step 3: Canny Edge Detection

# -------------------------

# Thresholds can be adjusted for sensitivity

canny_edges = cv2.Canny(img, threshold1=100, threshold2=200)


# -------------------------
```

```
# Step 4: Display Results
# -------------------------
plt.figure(figsize=(12,5))

plt.subplot(1,3,1)
plt.title("Original Image")
plt.imshow(img, cmap='gray')

plt.subplot(1,3,2)
plt.title("Sobel Edge Detection")
plt.imshow(sobel_combined, cmap='gray')

plt.subplot(1,3,3)
plt.title("Canny Edge Detection")
plt.imshow(canny_edges, cmap='gray')

plt.tight_layout()
plt.show()
```



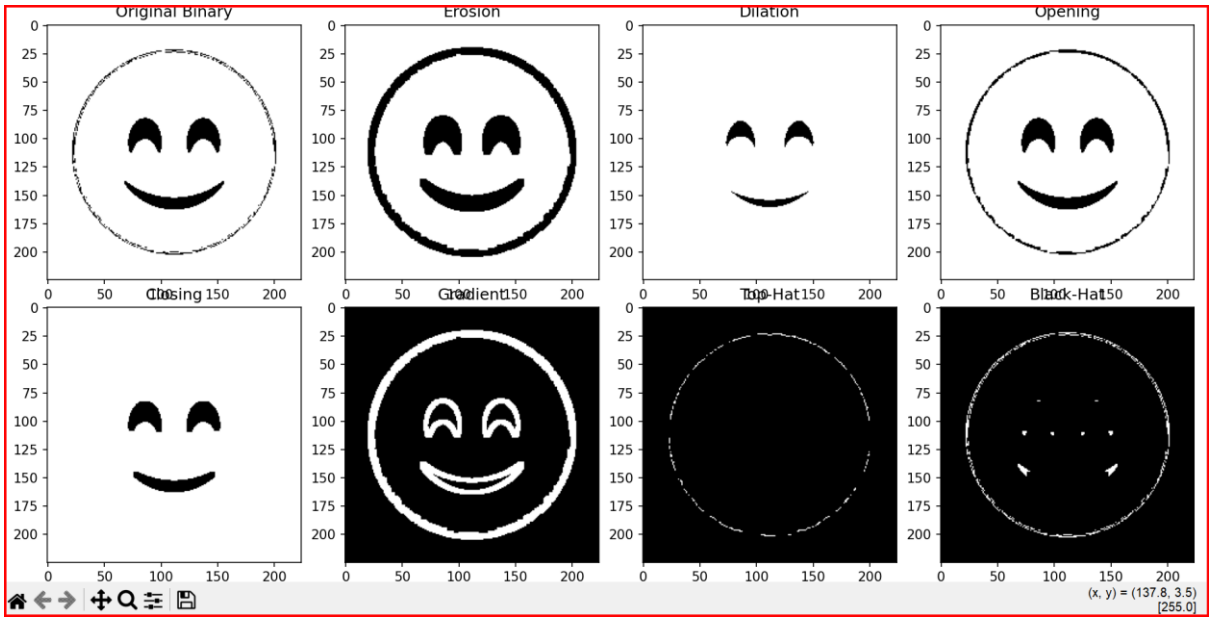
Practical 9

Write the program to implement various morphological image processing techniques.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# -------------------------
# Step 1: Load grayscale image
# -------------------------
img = cv2.imread("C:\\Users\\Shreyas S Sutar\\Desktop\MSC CS SEM-I\\download.jpg",
cv2.IMREAD_GRAYSCALE)

# Convert to binary for morphological operations
_, binary = cv2.threshold(img, 128, 255, cv2.THRESH_BINARY)

# Define a kernel (structuring element)
kernel = np.ones((5,5), np.uint8)

# -------------------------
# Step 2: Basic Morphological Operations
# -------------------------
erosion = cv2.erode(binary, kernel, iterations=1)
dilation = cv2.dilate(binary, kernel, iterations=1)
opening = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel)   # erosion followed by dilation
closing = cv2.morphologyEx(binary, cv2.MORPH_CLOSE, kernel)  # dilation followed by erosion
gradient = cv2.morphologyEx(binary, cv2.MORPH_GRADIENT, kernel)  # difference between dilation
and erosion
tophat = cv2.morphologyEx(binary, cv2.MORPH_TOPHAT, kernel)      # original - opening
blackhat = cv2.morphologyEx(binary, cv2.MORPH_BLACKHAT, kernel)  # closing - original

# -------------------------
```

```
# Step 3: Display Results
# -------------------------
plt.figure(figsize=(14,8))

plt.subplot(2,4,1)
plt.title("Original Binary")
plt.imshow(binary, cmap='gray')

plt.subplot(2,4,2)
plt.title("Erosion")
plt.imshow(erosion, cmap='gray')

plt.subplot(2,4,3)
plt.title("Dilation")
plt.imshow(dilation, cmap='gray')

plt.subplot(2,4,4)
plt.title("Opening")
plt.imshow(opening, cmap='gray')

plt.subplot(2,4,5)
plt.title("Closing")
plt.imshow(closing, cmap='gray')

plt.subplot(2,4,6)
plt.title("Gradient")
plt.imshow(gradient, cmap='gray')

plt.subplot(2,4,7)
plt.title("Top-Hat")
plt.imshow(tophat, cmap='gray')
```

```
plt.subplot(2,4,8)
plt.title("Black-Hat")
plt.imshow(blackhat, cmap='gray')

plt.tight_layout()
plt.show()
```



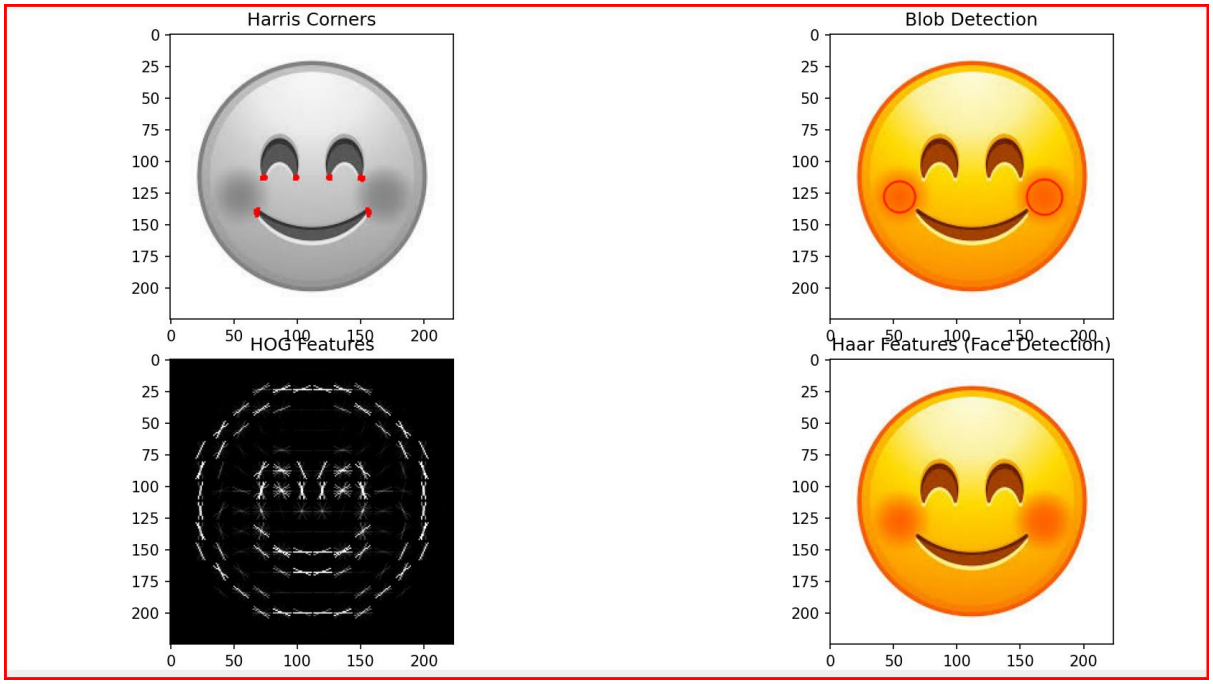
Practical 10.

Write the program to extract image features by implementing methods like corner and blob detectors, HoG and Haar features

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

from skimage.feature import hog

from skimage import exposure


# -------------------------

# Step 1: Load Image

# -------------------------

img = cv2.imread("C:\\Users\\Shreyas S Sutar\\Desktop\MSC CS SEM-I\\download.jpg")


# Convert to grayscale for some operations

if len(img.shape) == 3:  # color image

    img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

else:

    img_gray = img


# -------------------------

# Step 2: Harris Corner Detection

# -------------------------

corners = cv2.cornerHarris(img_gray, blockSize=2, ksize=3, k=0.04)

corners_dilated = cv2.dilate(corners, None)

corner_img = cv2.cvtColor(img_gray, cv2.COLOR_GRAY2BGR)

corner_img[corners_dilated>0.01*corners_dilated.max()] = [0,0,255]  # mark corners in red


# -------------------------

# Step 3: Blob Detection

# -------------------------
```

```
params = cv2.SimpleBlobDetector_Params()
params.filterByArea = True
params.minArea = 50
detector = cv2.SimpleBlobDetector_create(params)
keypoints = detector.detect(img_gray)
blob_img = cv2.drawKeypoints(img, keypoints, None, (0,0,255),
                             cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)

# -------------------------
# Step 4: HOG Feature Extraction
# -------------------------
hog_features, hog_image = hog(
    img_gray,
    pixels_per_cell=(16,16),
    cells_per_block=(2,2),
    visualize=True,
    channel_axis=None  # grayscale image
)
hog_image_rescaled = exposure.rescale_intensity(hog_image, in_range=(0, 10))

# -------------------------
# Step 5: Haar Feature Detection (Face Detection Example)
# -------------------------
haar_cascade = cv2.CascadeClassifier(cv2.data.haarcascades +
"haarcascade_frontalface_default.xml")
faces = haar_cascade.detectMultiScale(img_gray, scaleFactor=1.1, minNeighbors=5)
haar_img = img.copy()
for (x, y, w, h) in faces:
    cv2.rectangle(haar_img, (x, y), (x+w, y+h), (0,255,0), 2)

# -------------------------
```

```
# Step 6: Display Results
# -------------------------
plt.figure(figsize=(12,10))

plt.subplot(2,2,1)
plt.title("Harris Corners")
plt.imshow(corner_img[...,::-1])  # convert BGR to RGB for matplotlib

plt.subplot(2,2,2)
plt.title("Blob Detection")
plt.imshow(blob_img[...,::-1])

plt.subplot(2,2,3)
plt.title("HOG Features")
plt.imshow(hog_image_rescaled, cmap='gray')

plt.subplot(2,2,4)
plt.title("Haar Features (Face Detection)")
plt.imshow(haar_img[...,::-1])

plt.tight_layout()
plt.show()
```



Practical 11

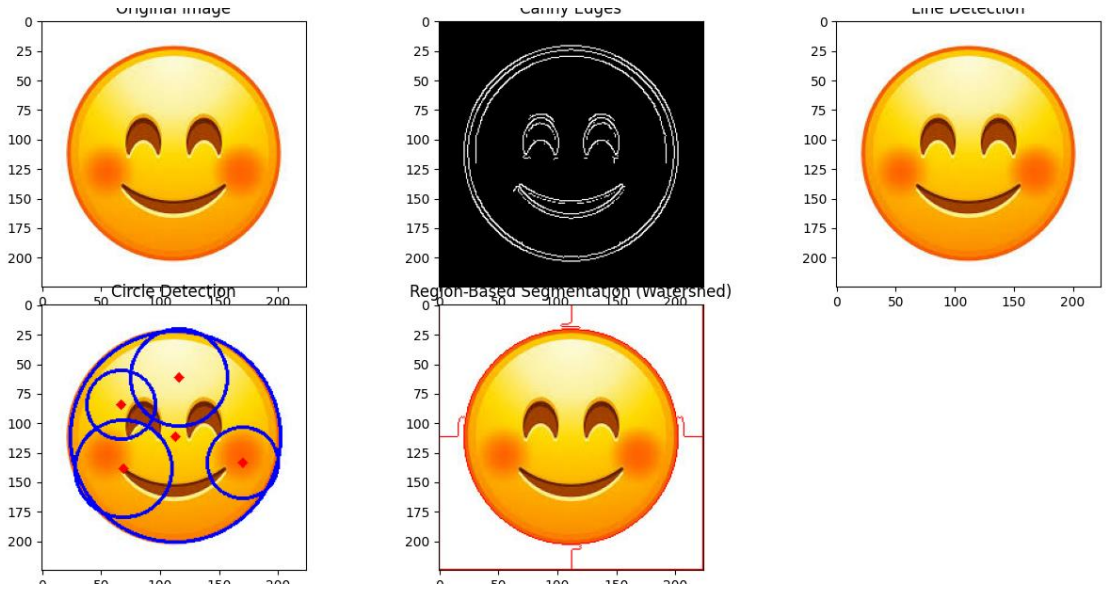
Aim :-Write the program to apply segmentation for detecting lines, circles, and other shapes/ objects. Also, implement edge-based and region-based segmentation.

```
import cv2

import numpy as np

import matplotlib.pyplot as plt


# -------------------------

# Step 1: Load Image

# -------------------------

img = cv2.imread("C:\\Users\\Shreyas S Sutar\\Desktop\MSC CS SEM-I\\download.jpg")

img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)


# -------------------------

# Step 2: Edge-Based Segmentation (Canny)

# -------------------------
```

```
edges = cv2.Canny(img_gray, 50, 150)

# -------------------------
# Step 3: Line Detection (Hough Transform)
# -------------------------
lines = cv2.HoughLinesP(edges, 1, np.pi/180, threshold=80, minLineLength=50, maxLineGap=10)
line_img = img.copy()
if lines is not None:
    for line in lines:
        x1, y1, x2, y2 = line[0]
        cv2.line(line_img, (x1, y1), (x2, y2), (0,255,0), 2)

# -------------------------
# Step 4: Circle Detection (Hough Transform)
# -------------------------
circles = cv2.HoughCircles(img_gray, cv2.HOUGH_GRADIENT, dp=1.2, minDist=50,
                           param1=50, param2=30, minRadius=10, maxRadius=100)
circle_img = img.copy()
if circles is not None:
    circles = np.uint16(np.around(circles))
    for i in circles[0,:]:
        cv2.circle(circle_img, (i[0], i[1]), i[2], (255,0,0), 2)
        cv2.circle(circle_img, (i[0], i[1]), 2, (0,0,255), 3)  # center

# -------------------------
# Step 5: Region-Based Segmentation (Watershed)
# -------------------------
# Thresholding
_, thresh = cv2.threshold(img_gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# Noise removal
```

```
kernel = np.ones((3,3), np.uint8)
opening = cv2.morphologyEx(thresh, cv2.MORPH_OPEN, kernel, iterations=2)

# Sure background and foreground
sure_bg = cv2.dilate(opening, kernel, iterations=3)
dist_transform = cv2.distanceTransform(opening, cv2.DIST_L2, 5)
_, sure_fg = cv2.threshold(dist_transform, 0.5*dist_transform.max(), 255, 0)

# Unknown region
sure_fg = np.uint8(sure_fg)
unknown = cv2.subtract(sure_bg, sure_fg)

# Marker labeling
_, markers = cv2.connectedComponents(sure_fg)
markers = markers + 1
markers[unknown==255] = 0

# Apply Watershed
watershed_img = img.copy()
markers = cv2.watershed(watershed_img, markers)
watershed_img[markers == -1] = [0,0,255]  # boundaries in red

# -------------------------
# Step 6: Display Results
# -------------------------
plt.figure(figsize=(14,10))

plt.subplot(2,3,1)
plt.title("Original Image")
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
```

```
plt.subplot(2,3,2)
plt.title("Canny Edges")
plt.imshow(edges, cmap='gray')

plt.subplot(2,3,3)
plt.title("Line Detection")
plt.imshow(cv2.cvtColor(line_img, cv2.COLOR_BGR2RGB))

plt.subplot(2,3,4)
plt.title("Circle Detection")
plt.imshow(cv2.cvtColor(circle_img, cv2.COLOR_BGR2RGB))

plt.subplot(2,3,5)
plt.title("Region-Based Segmentation (Watershed)")
plt.imshow(cv2.cvtColor(watershed_img, cv2.COLOR_BGR2RGB))

plt.tight_layout()
plt.show()
```


Original Image
Canny Edges
Line Detection
Circle Detection
Region-Based Segmentation (Watershed)

Explanation of Each practical

1. Upsampling/Downsampling & Fast Fourier Transform (FFT/DFT)

In this practical, you first **increase (upsample) or decrease (downsample) the resolution** of a signal or image. Upsampling adds more samples to make the signal smoother, while downsampling reduces samples, which may cause loss of information if not done carefully. You then use the **Fast Fourier Transform (FFT)** to convert the signal from the **time or spatial domain to the frequency domain**, which shows the different frequency components in the signal. This helps you **understand how signals are composed of frequencies**, detect patterns, and identify unwanted noise or interference. You also learn about **aliasing**, Nyquist rate, and the importance of proper sampling. Applications include **audio processing (like equalizers), image compression, and communication systems** where frequency analysis is critical.

2. Triangle Signal & Correlation

Here, you generate a **triangle waveform**, which is a simple periodic signal, and plot a segment covering multiple periods. This helps visualize **how periodic signals behave** over time. You also compute the **correlation between two signals**, which measures how similar they are. This is important because in real-world applications, you often need to **match or detect patterns in signals**, such as identifying repeated patterns in music, sensor data, or communication signals. Students learn about **periodicity, waveform characteristics, and similarity measurement**, which forms the basis for signal matching and detection.

3. Convolution & Template Matching

This practical introduces **convolution**, a fundamental operation in signal and image processing. Convolution allows you to **apply filters** like blurring, sharpening, or edge detection to images or signals. Template matching teaches how to **find a small pattern inside a larger image**, by sliding a template over the image and comparing intensities. Students learn how **feature extraction and filtering are performed**, and how patterns or objects are detected. This is widely used in **image recognition, surveillance, medical imaging, and object tracking** in computer vision.

4. Pixel Intensity Transformations

Students perform transformations like **log, power-law (gamma), contrast stretching, histogram equalization, thresholding, and halftoning**. These operations change the brightness or contrast of the image, enhance details, and adjust intensity levels for better visualization. For example, histogram equalization spreads out intensity levels to improve contrast. This practical teaches students **how to enhance images for human interpretation or machine processing**, and the concepts of **dynamic range, contrast, and intensity mapping**. Applications include **medical image enhancement, satellite imagery analysis, and printing technology**.

5. Gradient & Laplacian Operations

In this practical, students apply **derivative-based operators** to images. The **gradient** (first derivative) shows the rate of intensity change in specific directions, helping detect edges. The **Laplacian** (second derivative) highlights areas of rapid intensity change, detecting edges in all directions. Students learn how to extract **structural and edge information** from images, which is essential for **feature extraction, object detection, and image enhancement**. This is used in **autonomous vehicles, OCR, and industrial inspection**.

6. Linear & Nonlinear Noise Smoothing

Students apply **linear filters** (like averaging or Gaussian) and **nonlinear filters** (like median) to remove noise from images or audio signals. Linear filters smooth the image by averaging neighboring pixels, which is effective for **Gaussian noise**, while nonlinear filters like median preserve edges while removing **impulsive noise (salt-and-pepper noise)**. This teaches students how to **preprocess data**, reduce noise, and prepare signals or images for further analysis. Applications include **speech enhancement, digital photography, and medical imaging**.

7. Smoothing, Sharpening & Unsharp Masking

In this practical, students learn how to **enhance images**. Smoothing reduces noise and minor variations, sharpening highlights fine details, and **unsharp masking** enhances edges while keeping smooth areas intact. This helps in improving the **visual quality of images**, making details more prominent for analysis. Students learn **how filtering and derivatives can enhance visual perception**. Applications include **medical imaging, digital photography, and document clarity enhancement**.

8. Edge Detection (Sobel & Canny)

This practical demonstrates how to **detect edges** in images using **Sobel (gradient-based) and Canny (multi-stage) methods**. Edges represent significant intensity changes and often correspond to object boundaries. Students learn how edges are **detected, thresholded, and filtered**, and how noise affects the results. Edge detection is fundamental for **object recognition, segmentation, and machine vision** applications like lane detection in autonomous vehicles.

9. Morphological Image Processing

Morphological operations like **erosion, dilation, opening, closing, gradient, top-hat, and black-hat** help process shapes and structures in binary or grayscale images. Students learn how to **remove noise, fill holes, and enhance structural elements** in images. This practical demonstrates the role of morphology in **shape analysis, feature extraction, and preprocessing**. Applications include **OCR, industrial inspection, and preparing images for segmentation or pattern recognition**.

10. Feature Extraction (Corners, Blobs, HOG, Haar)

Students detect important points or features in images using **Harris corners, blobs, Histogram of Oriented Gradients (HOG), and Haar cascades**. Corners detect sharp changes in intensity, blobs detect bright or dark regions, HOG captures **shape and texture patterns**, and Haar detects objects using pre-trained classifiers. Students learn how **features describe an image's structure and are used for detection or recognition**. Applications include **face detection, object tracking, autonomous vehicles, and computer vision systems**.

11. Segmentation (Lines, Circles, Shapes) & Edge/Region-Based

In this practical, students segment images to **detect lines, circles, and other objects**. Edge-based methods like Canny and Hough detect boundaries, while region-based methods like Watershed segment **homogeneous regions**. Students learn how to **separate objects from the background and handle overlapping structures**. Applications include **traffic sign detection, cell segmentation in medical imaging, and industrial inspection**, where identifying objects accurately is crucial.